

Samsung Innovation Campus (SIC) Training

Capstone Project — Documentation

Customer Management: Churn Prediction Lakehouse Pipeline

Team	Abdullah Fathy, Salsabel Osama, Alaa Hany, Abdelrahman Adel
Facilitator	Abdelrahman Anwar
Date	September 5, 2026
Batch	802
Group	12

0. Introduction & Project Overview

This document is the technical documentation for the team's SIC Capstone project. It follows the project's key-consideration checklist (data sourcing, ingestion, processing, and warehouse loading) and records the design decisions, architecture, and status agreed on so far. Sections marked with a dashed box or "[TO BE FILLED]" are placeholders to be completed as the team progresses — for screenshots, query outputs, finalized code, and diagrams.

Section	Detail
Project Title	Customer Management (Churn Customer)
Objective	Build a data pipeline (NiFi/Sqoop → PySpark ETL) that cleans, enriches, transforms and aggregates raw customer data into a comprehensive Dim_Customer feature set for a churn-prediction ML model.
Target Architecture	Lakehouse pattern, built on Medallion Architecture (Bronze → Silver → Gold), following a Star Schema in the Gold layer.

1. Database & Data Sourcing

1.1 Data Structure Understanding

The source data is organized around four simulated tables, each covering a different aspect of the customer relationship:

Table Name	Format / Source	Key Column	Example Attributes
Customers	RDBMS / Sqoop	customer_id (PK)	age, gender, geography, is_active, tenure_months, date_opened
Customer_support_tickets	CSV / Batch	ticket_id (PK)	customer_id (FK), issue_type, severity, resolution_time_hrs
Offers	CSV / Batch	offer_id (PK)	customer_id (FK), offer_type, accepted, date_offered
Usage	JSON / Log	usage_log_id (PK)	customer_id (FK), product_type, monthly_balance, num_products

All three child tables (support tickets, offers, usage) relate back to Customers through the customer_id foreign key, which is the join key used throughout the pipeline.

1.2 Relevant Data Identification

- Customers — the core entity and source of the churn label (is_active / churn history) and demographic/profile features.
- Customer_support_tickets — engagement/friction signal; feeds the avg_ticket_res_time_hrs feature and can indicate dissatisfaction ahead of churn.
- Offers — marketing engagement signal; whether a customer accepted retention/upsell offers.
- Usage — behavioral and financial signal; feeds CLV_LTV and num_products.

No additional tables are planned beyond these four sources, to keep the pipeline scoped to what directly supports the Dim_Customer feature set and churn label.

2. Data Ingestion with Data Engineering Tools

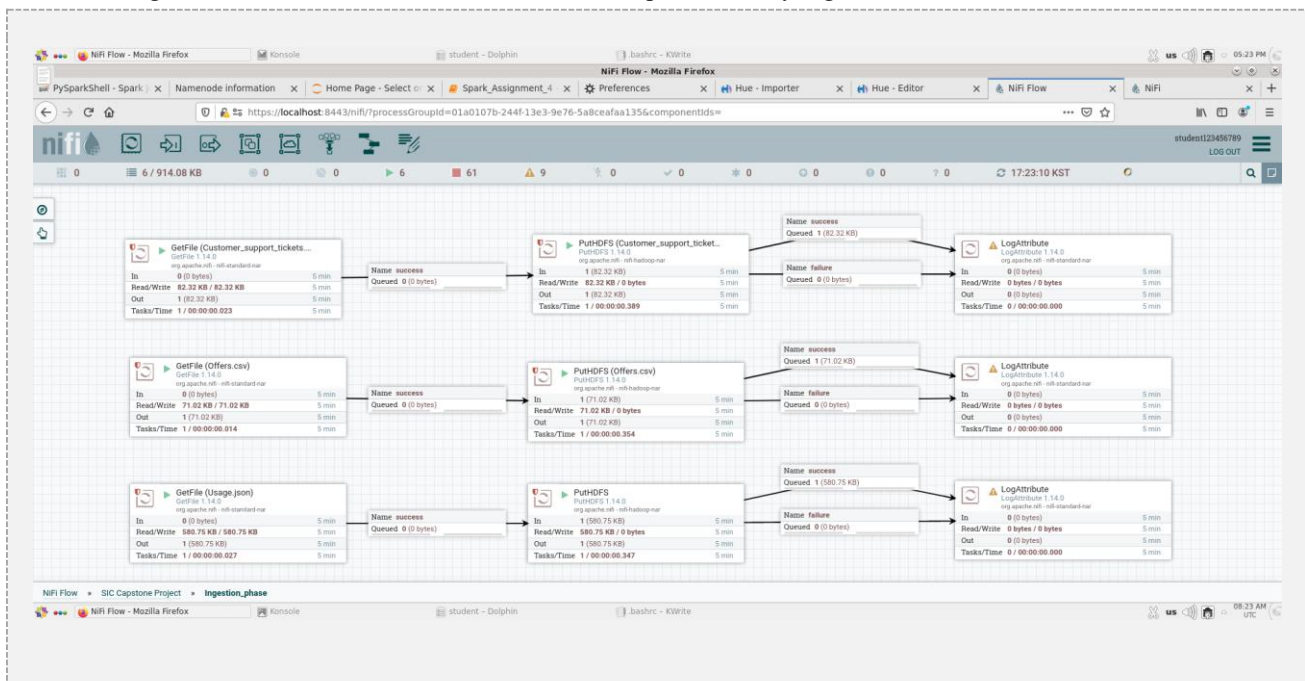
2.1 Tool Selection

Source	Tool	Reason
Customers (RDBMS)	Apache Sqoop	Relational source with a stable schema; Sqoop gives efficient, parallel snapshot imports into HDFS.
Support tickets, Offers (CSV)	Apache NiFi	File-based batch sources; NiFi handles pickup, basic validation/routing, and lands files in Bronze.
Usage (JSON log)	Apache NiFi	Simulates a streaming/log-style source; NiFi's flow-based model fits log ingestion best.

2.2 Ingestion Pipeline Design

All sources land first in the Bronze layer of HDFS, preserving raw structure. Customers is ingested as a batch snapshot via Sqoop; support tickets, offers, and usage logs flow through NiFi, which will also be responsible for the presenting/build of the full end-to-end pipeline (see Section 6).

- Batch vs. streaming: all four sources are treated as batch loads for this project; the Usage/JSON source simulates streaming but is processed in scheduled batches, given the project timeline.
- Landing zone: raw files/tables land in HDFS Bronze, partitioned by ingestion date.



2.3 Error Handling

- NiFi: use failure relationships on each processor to route bad/unreadable records to a dedicated error queue instead of dropping them silently.
- Sqoop: rely on job-level retry and logging for connection interruptions; failed imports are logged with the source table and timestamp.
- A short ingestion report should record: records received, records successfully landed, records failed/quarantined, and reason categories.

3. Data Processing & Transformation with Spark

3.1 Schema Design (Star Schema)

The Gold layer follows a Star Schema. Dim_Customer is the primary dimension table carrying churn-relevant features; a fact table aggregating usage/interactions at the customer level supports it.

Column	Derivation / Notes	Role
cust_key	Surrogate key (sequence generated)	Primary Key (PK) for DW
customer_id	Business key from Customers	Business Key (BK)
is_churned	Derived from historical is_active status	Target Variable
CLV_LTV	Aggregated Usage and transactions (simulated)	Enrichment Feature
avg_ticket_res_time_hrs	Aggregation from Customer_support_tickets	Engagement Feature
num_products	Current number of products held	Profile Feature
dw_start_date	Start date of the current dimensional record	SCD tracking
dw_end_date	End date of the current record (e.g., '9999-12-31' if current)	SCD tracking

3.2 Data Transformations by Layer

Layer	Transformations
Bronze → Silver	Cleaning: remove/flag missing values, correct type mismatches, standardize categorical spellings (e.g., issue_type, geography). Deduplication of records by primary key. Save as Parquet.
Silver → Gold	Enrichment: join Customers with Usage, Offers, and Support tickets on customer_id. Aggregation: compute CLV_LTV, avg_ticket_res_time_hrs, num_products, and derive is_churned from historical is_active status. Load into the Dim_Customer star-schema table (and companion fact table).

3.3 Spark Performance Considerations

- Prefer DataFrame/Dataset aggregations (groupBy().agg(...)) over RDD-level groupByKey, which forces a full shuffle of all values.
- Where RDD-style aggregation is unavoidable, use reduceByKey / aggregateByKey instead of groupByKey to reduce shuffle volume.
- Cache/persist intermediate DataFrames that are reused across multiple downstream transformations (e.g., the cleaned Silver Customers DataFrame).
- Monitor the Spark UI for wide-shuffle stages when joining the four sources and adjust partitioning (e.g., repartition by customer_id) if skew appears.

```
NRF Flow - Mozilla Firefox  student - Dolphin  bashrc - KWrite
student: bash - Konsole <2>

[hadoop@localhost Desktop]$ python3 -m py_compile ~/Desktop/silver_to_gold.py
[hadoop@localhost Desktop]$ export SAFE_MODE=0
[hadoop@localhost Desktop]$ export DEBUG=0
[hadoop@localhost Desktop]$ /usr/local/spark3/spark-3.1.2-bin-hadoop3.2/bin/spark-submit \
  -j ~/Desktop/silver_to_gold.py 2m1 | tee ~/Desktop/gold_run.log
2026-09-04 18:15:33.642 WARN util.Utils: Your hostname, localhost.localdomain resolves to a loopback address: 127.0.0.1; using 192.168.1.16 instead (on interface ens33)
2026-09-04 18:15:33.647 WARN util.Utils: Set SPARK_LOCAL_IP if you need to bind to another address
2026-09-04 18:15:34.964 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-09-04 18:15:36.355 INFO spark.SparkContext: Running Spark version 3.1.2
2026-09-04 18:15:36.501 INFO resource.ResourceUtils: No custom resources configured for spark.driver.
2026-09-04 18:15:36.584 INFO resource.ResourceUtils:
=====
2026-09-04 18:15:36.585 INFO spark.SparkContext: Submitted application: Churn Silver To Gold
2026-09-04 18:15:36.587 INFO resource.ResourceProfile: Default ResourceProfile created, executor resources: Map(cores -> name: cores, amount: 1, script: , vendor: , memory -> name: memory, amount: 1024, script: , vendor: , offHeap -> name: offHeap, amount: 0, script: , vendor: ), task resources: Map(cpu -> name: cpus, amount: 1.0)
2026-09-04 18:15:36.618 INFO resource.ResourceProfile: Limiting resource is cpu
2026-09-04 18:15:36.620 INFO resource.ResourceProfileManager: Added ResourceProfile id: 0
2026-09-04 18:15:36.782 INFO spark.SecurityManager: Changing view acls to: hadoop
2026-09-04 18:15:36.783 INFO spark.SecurityManager: Changing modify acls to: hadoop
2026-09-04 18:15:36.783 INFO spark.SecurityManager: Changing view acls groups to:
2026-09-04 18:15:36.783 INFO spark.SecurityManager: Changing modify acls groups to:
2026-09-04 18:15:36.783 INFO spark.SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissions: Set(hadoop); groups with view permissions: Set(); users with modify permissions: Set(hadoop); groups with modify permissions: Set()
2026-09-04 18:15:37.592 INFO util.Utils: Successfully started service 'sparkDriver' on port 43239.
2026-09-04 18:15:37.674 INFO spark.SparkEnv: Registering MapOutputTracker
2026-09-04 18:15:37.712 INFO spark.SparkEnv: Registering BlockManagerMaster
2026-09-04 18:15:37.687 INFO storage.BlockManagerMasterEndpoint: Using org.apache.spark.storage.DefaultTopologyMapper for getting topology information
2026-09-04 18:15:37.888 INFO storage.BlockManagerMasterEndpoint: BlockManagerMasterEndpoint up
2026-09-04 18:15:37.890 INFO spark.SparkEnv: Registering BlockManagerMasterHeartbeat
2026-09-04 18:15:37.922 INFO storage.DiskBlockManager: Created local directory at /tmp/blockmgr-25acf6-85fd-423a-b9e6-348a9189a39b
2026-09-04 18:15:37.985 INFO memory.MemoryStore: MemoryStore started with capacity 366.3 MiB
2026-09-04 18:15:38.439 INFO spark.SparkEnv: Registering OutputCommitCoordinator
2026-09-04 18:15:38.921 INFO util.Log: Logging initialized @7934ms to org.apache.spark.util.log.Slf4jLog
2026-09-04 18:15:38.638 INFO server.Server: jetty-9.4.40.v20210413; built: 2021-04-13T20:42:42.668Z; git: b881a572662e1943a14ae12e7e1207989f218b74; jvm 1.8.0_291-b10
2026-09-04 18:15:38.725 INFO server.Server: Started @824ms
2026-09-04 18:15:38.893 INFO server.AbstractConnector: Started ServerConnector@4b3d8308(HTTP/1.1, {http/1.1}){0.0.0.0:4040}
2026-09-04 18:15:38.854 INFO util.Utils: Successfully started service 'SparkUI' on port 4040.
2026-09-04 18:15:38.953 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@5af85876(/jobs,null,AVAILABLE,@Spark)
2026-09-04 18:15:38.960 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@39d5715f(/jobs/json,null,AVAILABLE,@Spark)
2026-09-04 18:15:38.961 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@6bb5c57f3(/jobs/job,null,AVAILABLE,@Spark)
2026-09-04 18:15:38.974 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@2e38c467(/jobs/job/json,null,AVAILABLE,@Spark)
2026-09-04 18:15:38.972 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@215d6b5e(/stages/json,null,AVAILABLE,@Spark)
2026-09-04 18:15:38.977 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@7ece2978(/stages/job,null,AVAILABLE,@Spark)
```

4. Data Warehouse Loading

4.1 Loading Tool

Spark writes the final Gold-layer tables directly to Hive external tables.

4.2 File Format

Parquet is used for all Silver and Gold layer storage, for compression and columnar-scan performance on analytical queries. Bronze layer retains the original raw format (CSV/JSON) as landed.

4.3 Partitioning Strategy

- Dim_Customer: consider partitioning by geography or by dw_start_date (year/month) if the table grows large enough to benefit.
- Fact table (usage/interactions): partition by date, since time-range queries (e.g., monthly churn rate) are the primary access pattern.

4.4 Hive DDL

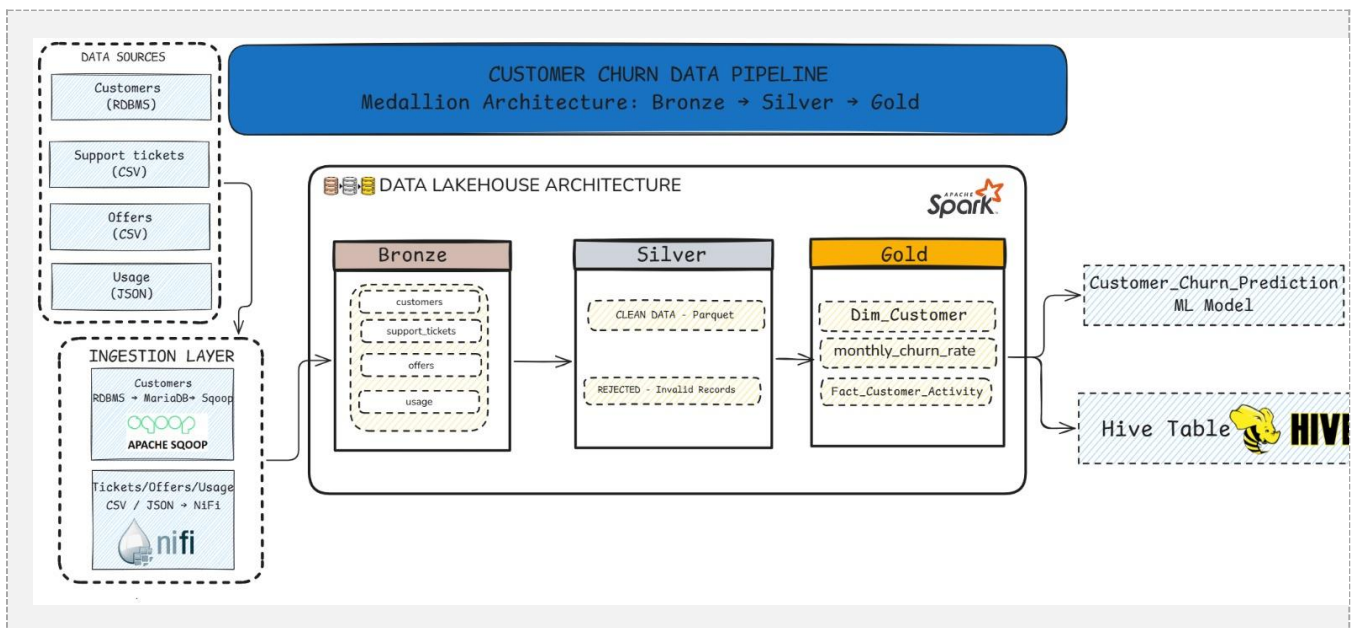
Space reserved for the finalized Hive DDL for Dim_Customer (and the companion fact table).

The screenshot shows the Hive web interface. On the left, a sidebar lists tables under 'churn_gold': dim_customer_gold, fact_customer_activity_gold, and monthly_churn_rate_gold. The main area displays a SQL query with line numbers 97 to 114. The query uses UNION ALL to combine counts from three tables. Below the query, the 'Results (3)' tab is active, showing a table with two columns: '_u1.table_name' and '_u1.row_count'. The results are as follows:

_u1.table_name	_u1.row_count
fact_customer_activity	3278
monthly_churn_rate	23
dim_customer	684

5. Architecture We Will Build

The overall design is a Lakehouse built on Medallion Architecture. The Data Architect (Alaa Hany) is mapping the full lifecycle in Excalidraw, with each stage documented as a set of rules describing its exact function. The diagram below is a placeholder for that architecture export.



High-level flow (to be refined in the diagram above):

- Sources: Customers (RDBMS), Support Tickets (CSV), Offers (CSV), Usage (JSON)
- Ingestion: Sqoop (Customers) + NiFi (CSV/JSON sources) → HDFS Bronze
- Bronze → Silver: PySpark cleaning & validation → Parquet
- Silver → Gold: PySpark modeling, enrichment & aggregation → Gold layer containing Dim_Customer, Fact_Customer_Activity, and monthly_churn_rate tables (Hive/Parquet, optionally Iceberg)
- Consumption: HiveQL / Presto for KPI and monthly churn-rate analysis using the Gold-layer tables; optional ML Model (bonus, TBD) on top of Gold

6. Tools and Technologies Applied

Category	Tool(s)
Ingestion	Apache NiFi (CSV/JSON sources), Apache Sqoop (relational snapshot of Customers)
Storage	Hadoop Distributed File System (HDFS) — data lake / Bronze-Silver-Gold zones
Processing	PySpark — Bronze→Silver cleaning, Silver→Gold modeling/enrichment (ETL jobs)
Warehousing	Apache Hive external tables
Data Modeling	Star Schema — Dim_Customer (dimension) + companion fact table (TBD)
SQL / Querying	HiveQL / Presto for KPI queries, including the monthly churn rate
ML Model (bonus)	ML Model — Churn prediction model built using the engineered customer features

7. Team & Role Assignments

Step	Owner	Stage / Focus	What Happens / Deliverable
1	Alaa Hany	Architecture Design	Maps the full data lifecycle in Excalidraw by Medallion stage within a Lakehouse model; documents rules per stage and business-facing metadata.
2	Abdelrahman Adel	Bronze → Silver	Ingests raw data into Bronze; builds cleaning/validation transformation into Silver (Parquet); produces a data-quality report.
3	Salsabel Osama	Silver → Gold	Builds modeling/transformation logic from Silver to Gold; produces a before/after data-shape report.
3.5	All Members	Local Sample Testing	Each member tests their stage locally and independently on a data sample before full integration.
4	Abdullah Fathy	Full Pipeline in NiFi	Builds the complete pipeline structure in NiFi, integrates all stages on the full dataset, and produces an infrastructure performance report.
Bonus	Alaa Hany	ML Model (optional)	ML Model — Churn prediction model built using the engineered customer features from Dim_Customer and Fact_Customer_Activity

8. Project Deliverables

- NiFi template and Sqoop scripts for ingestion
- PySpark ETL script (Bronze → Silver → Gold)
- Hive DDL for the final Dim_Customer table
- Spark SQL / HiveQL query to calculate the Monthly Churn Rate

- Architecture diagram (Excalidraw export)
- Data-quality and transformation reports (Bronze→Silver, Silver→Gold)
- Final presentation summarizing architecture, workflow, and results
- *ML Model (bonus, optional)*

9. Technology Stack & Purpose

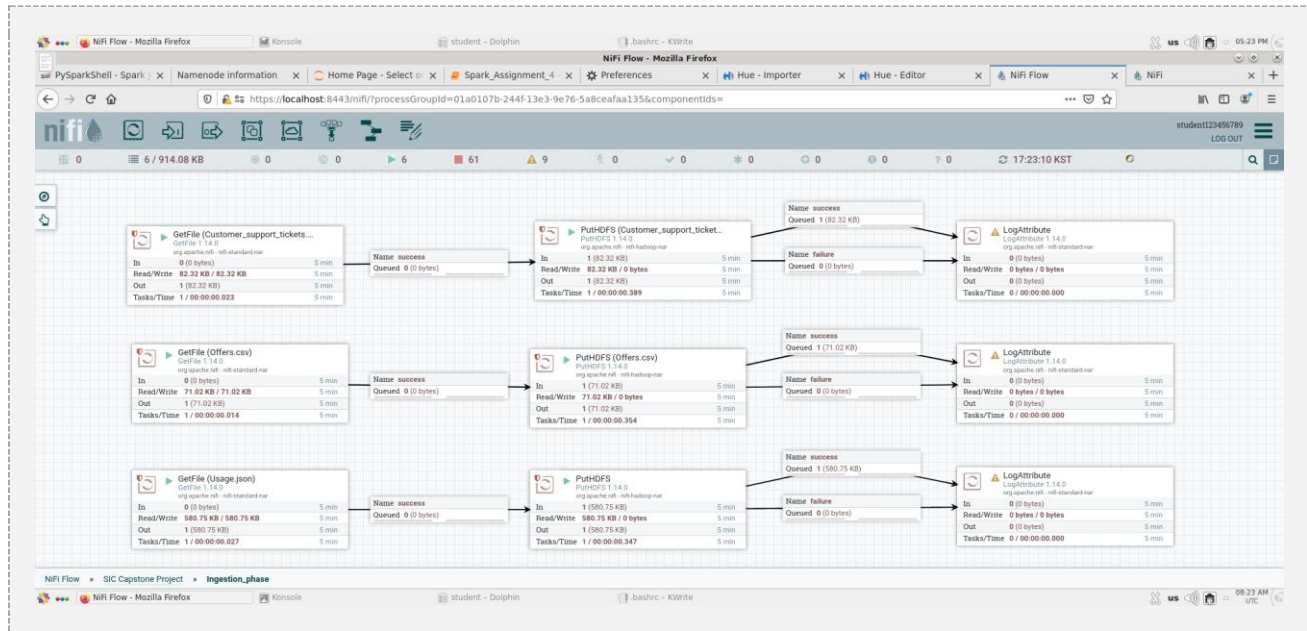
Tool / Technology	Purpose in the Project
Apache NiFi	Data ingestion and flow management from different data sources.
Apache Sqoop	Ingesting structured data from relational databases into Hadoop.
Apache Hadoop / HDFS	Storing large volumes of raw and processed data in a distributed environment.
Apache Spark / PySpark	Data processing, cleaning, transformation, and feature preparation for analytics and ML.
Apache Hive	Creating and managing the Data Warehouse layer and querying the processed data using SQL.

10. Appendix: Screenshots & Supporting Material

A. Data Exploration

***	RowNumber CustomerId Surname CreditScore Geography Gender Age Tenure Balance NumOfProducts HasCrCard IsActiveMember EstimatedSalary Exited tenure_months date_opened
	1 15634602 Hargrave 619 France Female 42 2 0.0 1 1 1 101348.88 1 24 2023-10-02
	2 15647311 Hill 608 Spain Female 41 1 83807.86 1 0 1 112542.58 0 12 2024-10-01
	3 15619304 Onio 502 France Female 42 8 159660.8 3 1 0 113931.57 1 96 2017-10-01
	4 15701354 Boni 699 France Female 39 1 0.0 2 0 0 93826.63 0 12 2024-10-01
	5 15737888 Mitchell 850 Spain Female 43 2 125510.82 1 1 1 79084.1 0 24 2023-10-02
only showing top 5 rows	
	offer_id customer_id offer_type accepted date_offered
	OFR79656043 15634602 card_upgrade 1 2024-12-01
	OFR72763121 15647311 fee_waiver 1 2025-08-02
	OFR78502572 15619304 card_upgrade 0 2024-04-02
	OFR85577339 15619304 loan_rate_cut 0 2025-06-02
	OFR80341070 15701354 card_upgrade 1 2024-09-01
only showing top 5 rows	
	ticket_id customer_id Issue_type severity resolution_time_hrs created_at
	TKT29044232 15634602 app_bug medium 1.28 2025-10-01
	TKT73000913 15647311 app_bug low 55.46 2025-10-01
	TKT13669663 15647311 app_bug low 9.89 2024-12-01
	TKT91237889 15619304 fee_dispute low 20.21 2024-09-01
	TKT63090346 15619304 fee_dispute medium 2.56 2024-08-01
only showing top 5 rows	
	customer_id monthly_balance num_products product_type usage_log_id usage_month
	15634602 0.0 1 investment USG948084549 2025-01
	15634602 48.47 1 mortgage USG637997907 2025-08
	15647311 82500.78 1 current_account USG227311678 2025-09
	15647311 72764.1 2 credit_card USG796030080 2025-07
	15647311 101956.38 2 credit_card USG973774504 2025-09
only showing top 5 rows	

B. NiFi Ingestion Flow



C. Spark Job Runs

```
student: bash - Konsole <2>
[hadoop@localhost Desktop]s grep -E "GOLD LAYER STATUS|ROW COUNTS|NULL|Duplicate|Orphan|Invalid|Negative|ERROR|Exception" \
~/Desktop/gold_run.log
2026-09-04 18:15:38.993 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@5af65876(/jobs,null,AVAILABLE,@spark)
2026-09-04 18:15:38.968 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@39d57151(/jobs/json,null,AVAILABLE,@spark)
2026-09-04 18:15:38.974 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@6b6c5773(/jobs/job,null,AVAILABLE,@spark)
2026-09-04 18:15:38.975 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@23bca407(/jobs/job/json,null,AVAILABLE,@spark)
2026-09-04 18:15:38.975 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@215deb5e(/stages,null,AVAILABLE,@spark)
2026-09-04 18:15:38.977 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@7ece2978(/stages/json,null,AVAILABLE,@spark)
2026-09-04 18:15:38.978 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@7232329(/stages/stage,null,AVAILABLE,@spark)
2026-09-04 18:15:38.983 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@36f62241(/stages/stage/json,null,AVAILABLE,@spark)
2026-09-04 18:15:38.984 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@5d4764b0(/stages/pool,null,AVAILABLE,@spark)
2026-09-04 18:15:38.985 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@7da78b63(/stages/pool/json,null,AVAILABLE,@spark)
2026-09-04 18:15:38.986 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@8443a598(/storage,null,AVAILABLE,@spark)
2026-09-04 18:15:38.987 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@15d3d4d7(/storage/json,null,AVAILABLE,@spark)
2026-09-04 18:15:38.988 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@8391537b(/storage/rd,null,AVAILABLE,@spark)
2026-09-04 18:15:38.989 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@4a4d7b35(/storage/rd/json,null,AVAILABLE,@spark)
2026-09-04 18:15:38.989 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@121b083f(/environment,null,AVAILABLE,@spark)
2026-09-04 18:15:38.990 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@5da79fa8(/environment/json,null,AVAILABLE,@spark)
2026-09-04 18:15:38.991 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@38f569c(/executors,null,AVAILABLE,@spark)
2026-09-04 18:15:38.993 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@4b04e141(/executors/json,null,AVAILABLE,@spark)
2026-09-04 18:15:38.994 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@28b27894(/executors/threadDump,null,AVAILABLE,@spark)
2026-09-04 18:15:38.996 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@15451922(/executors/threadDump/json,null,AVAILABLE,@spark)
2026-09-04 18:15:39.033 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@951638e4(/static,null,AVAILABLE,@spark)
2026-09-04 18:15:39.033 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@5ab90b72(/,null,AVAILABLE,@spark)
2026-09-04 18:15:39.037 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@8a060ab(/api,null,AVAILABLE,@spark)
2026-09-04 18:15:39.038 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@26a26260(/jobs/job/kill,null,AVAILABLE,@spark)
2026-09-04 18:15:39.039 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@594b5668(/stages/stage/kill,null,AVAILABLE,@spark)
2026-09-04 18:15:40.375 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@75b93c8c(/metrics/json,null,AVAILABLE,@spark)
2026-09-04 18:15:41.277 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@73748c21(/SQL,null,AVAILABLE,@spark)
2026-09-04 18:15:41.278 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@4d447645(/SQL/json,null,AVAILABLE,@spark)
2026-09-04 18:15:41.280 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@45ba7381(/SQL/execution,null,AVAILABLE,@spark)
2026-09-04 18:15:41.281 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@6a4b08b1(/SQL/execution/json,null,AVAILABLE,@spark)
2026-09-04 18:15:41.297 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@6083fb04(/static/sql,null,AVAILABLE,@spark)
Silver row counts:
STEP 16 - Gold Row Counts
ROW COUNTS
NULL customer_id: 0
NULL cust key: 0
Duplicate cust key: 0
Duplicate customer_id: 0
Invalid is churned: 0
Negative CV LTV: 0
Negative avg ticket resolution: 0
NULL activity key: 0
```

```
Silver row counts:
STEP 16 - Gold Row Counts
ROW COUNTS
NULL customer_id: 0
NULL cust_key: 0
Duplicate cust_key: 0
Duplicate customer_id: 0
Invalid is_churned: 0
Negative CLV_LTV: 0
Negative avg_ticket_resolution: 0
NULL activity_key: 0
Duplicate activity_key: 0
Duplicate customer + month: 0
NULL customer_id: 0
NULL cust_key: 0
Orphan fact rows: 0
Invalid activity_month: 0
Negative usage_events: 0
Negative total_balance: 0
Negative ticket_count: 0
Negative offer_count: 0
Negative accepted_offers: 0
Invalid acceptance_rate: 0
Invalid churn_rate: 0
GOLD LAYER STATUS: PASS
[hadoop@localhost Desktop]$ C
```

```
[hadoop@localhost Desktop]$ hdfs dfs -ls /data
Found 4 items
drwxr-xr-x - hadoop supergroup      0 2026-09-04 17:40 /data/_staging
drwxrwxr-x - root supergroup        0 2026-09-04 17:21 /data/bronze
drwxr-xr-x - hadoop supergroup      0 2026-09-04 18:16 /data/gold
drwxr-xr-x - hadoop supergroup      0 2026-09-04 18:06 /data/silver
[hadoop@localhost Desktop]$ hdfs dfs -ls /data/gold
Found 3 items
drwxr-xr-x - hadoop supergroup      0 2026-09-04 18:16 /data/gold/dim_customer
drwxr-xr-x - hadoop supergroup      0 2026-09-04 18:16 /data/gold/fact_customer_activity
drwxr-xr-x - hadoop supergroup      0 2026-09-04 18:16 /data/gold/monthly_churn_rate
[hadoop@localhost Desktop]$
```

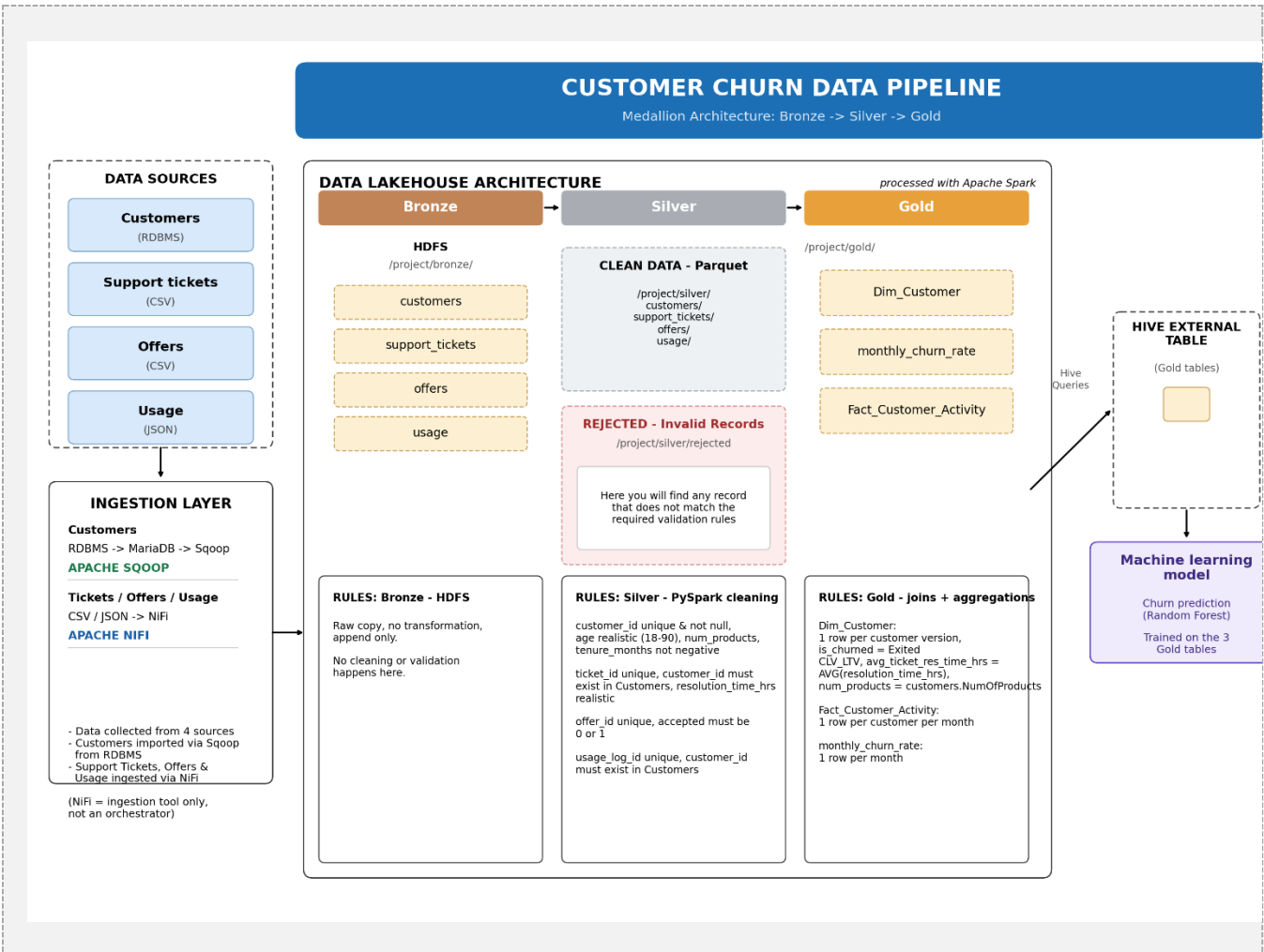
D. Hive Tables

The screenshot shows the Hive web interface. On the left, a sidebar lists tables under the 'churn_gold' database: 'dim_customer_gold', 'fact_customer_activity_gold', and 'monthly_churn_rate_gold'. The main area displays a SQL query that counts rows for three tables and unions the results. Below the query, the 'Results (3)' tab is active, showing a table with 3 rows and 2 columns: '_u1.table_name' and '_u1.row_count'.

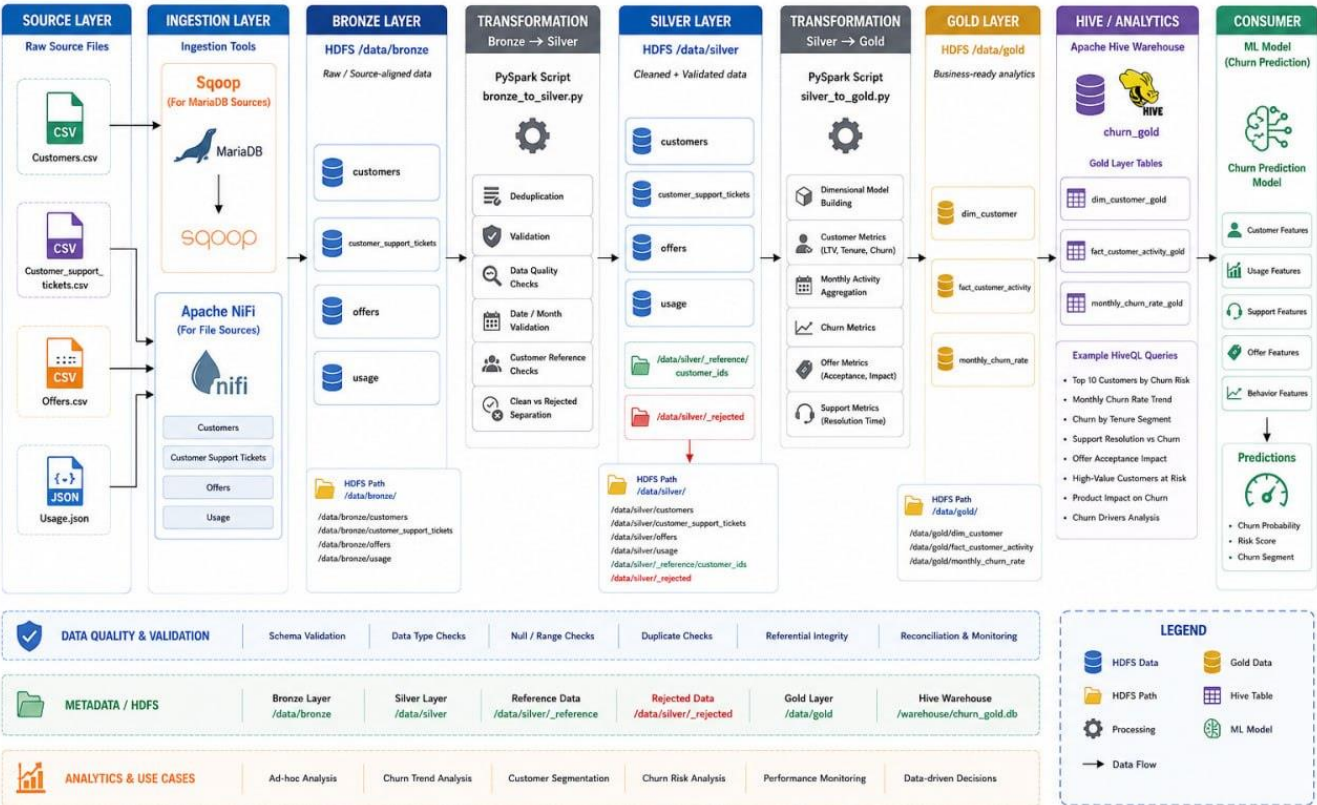
```
97 SELECT
98     'dim_customer' AS table_name,
99     COUNT(*) AS row_count
100 FROM dim_customer_gold
101 UNION ALL
102 SELECT
103     'fact_customer_activity',
104     COUNT(*)
105 FROM fact_customer_activity_gold
106 UNION ALL
107 SELECT
108     'monthly_churn_rate',
109     COUNT(*)
110 FROM monthly_churn_rate_gold;
```

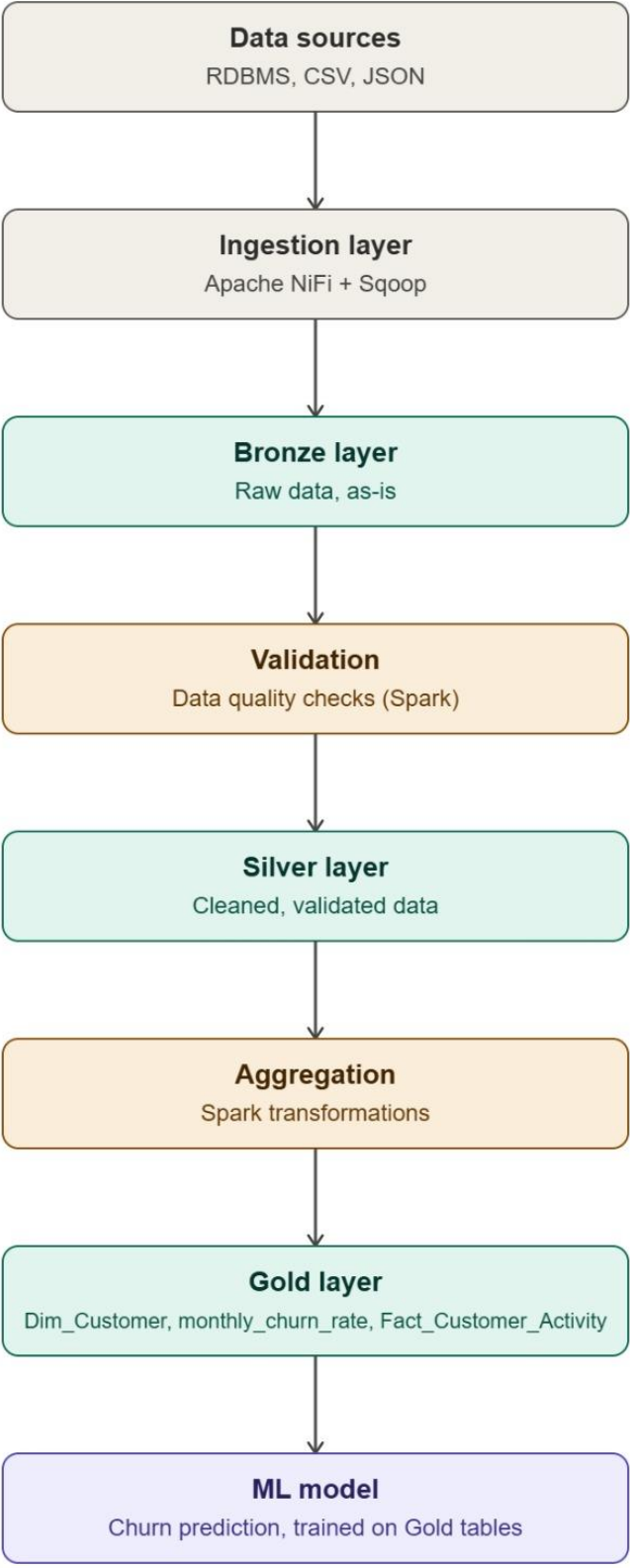
_u1.table_name	_u1.row_count
fact_customer_activity	3278
monthly_churn_rate	23
dim_customer	684

E. Architecture Diagram



CUSTOMER CHURN ETL PIPELINE





F. ML Model (Bonus)

9a84dec09bbe485223.gradio.live

Customer Churn Prediction

Enter a customer's feature values to estimate their churn probability.

num_products

10

CLV_LTV

456

avg_total_balance

345

avg_balance_mean

10000

avg_ticket_res_time_hrs

7

Prediction

Likely to churn
Churn probability: 68.22%

Flag

Clear

Submit